



Rapport de Travaux Pratiques

Contrôle de LEDs par PWM avec bouton

Gestion appui court / long sur ATmega2560

Environnement : Atmel Studio

Professeur : Mr BARRO

Réalisé par : Payié Mariane ELOLA

Sephora Maurane BAYALA

Mamadou Latif SALIFOU HAMANI

Année Académique : 2025-2026

SOMMAIRE

1.	INTRODUCTION.....	3
2.	Matériels et environnement logiciel.....	3
2.1.	Composants matériels.....	3
2.2.	Affectation des ports	3
2.3.	Environnement logiciel	3
3.	Montage.....	4
4.	Logigramme	5
5.	Déclarations des variables et constantes	6
6.	Gestion du PWM (Timer 1)	6
6.1.	Initialisation pwm_init().....	6
6.2.	Fonctions set_white(), set_yellow()	7
7.	Compteur millis() (Timer 3).....	7
8.	Lecture du bouton.....	7
9.	Logique applicative	7
9.1.	Appui court : changement d'état.....	7
9.2.	Appui long : variation de luminosité.....	8
10.	Programme principal et boucle de mesure.....	8
11.	Conclusion.....	10

1. INTRODUCTION

Ce travail pratique consiste à piloter deux LEDs (une blanche et une jaune) à l'aide d'un unique bouton poussoir connecté à un microcontrôleur ATmega2560. Le programme gère deux types d'appuis :

- Appui court : fait défiler les états de la lampe (éteinte → LED blanche → LED jaune → éteinte).
- Appui long : fait varier la luminosité de la LED active en temps réel par modulation de largeur d'impulsion (PWM).

Le programme est organisé en plusieurs blocs fonctionnels :

- `pwm_init()`, `set_white()`, `set_yellow()` : génération et contrôle du PWM sur Timer 1.
- `timer3_init()`, `millis()` : compteur de millisecondes sur Timer 3 pour mesurer les durées d'appui.
- `btn_init()`, `btn_read()` : configuration et lecture du bouton avec anti-rebond implicite.
- `applyLEDs()` et boucle principale : logique de décision et mise à jour des LEDs.

2. Matériels et environnement logiciel

2.1. Composants matériels

Composant	Référence / Valeur	Rôle
Microcontrôleur	ATmega2560	Unité centrale de traitement
LED blanche	PB5 (OC1A) — 5 mm	Indicateur état 1 — PWM Timer 1
LED jaune	PB6 (OC1B) — 5 mm	Indicateur état 2 — PWM Timer 1
Bouton poussoir	PE4 — NO + pull-up	Entrée utilisateur appui court/long
Résistance LED blanche	220 Ω	Limitation du courant dans la LED
Résistance LED jaune	220 Ω	Limitation du courant dans la LED
Alimentation	5 V DC (USB/AVRISP)	Alimentation microcontrôleur
Fils de connexion	Jumper wires	Connexions entre composants
Breadboard	830 points	Support de câblage sans soudure

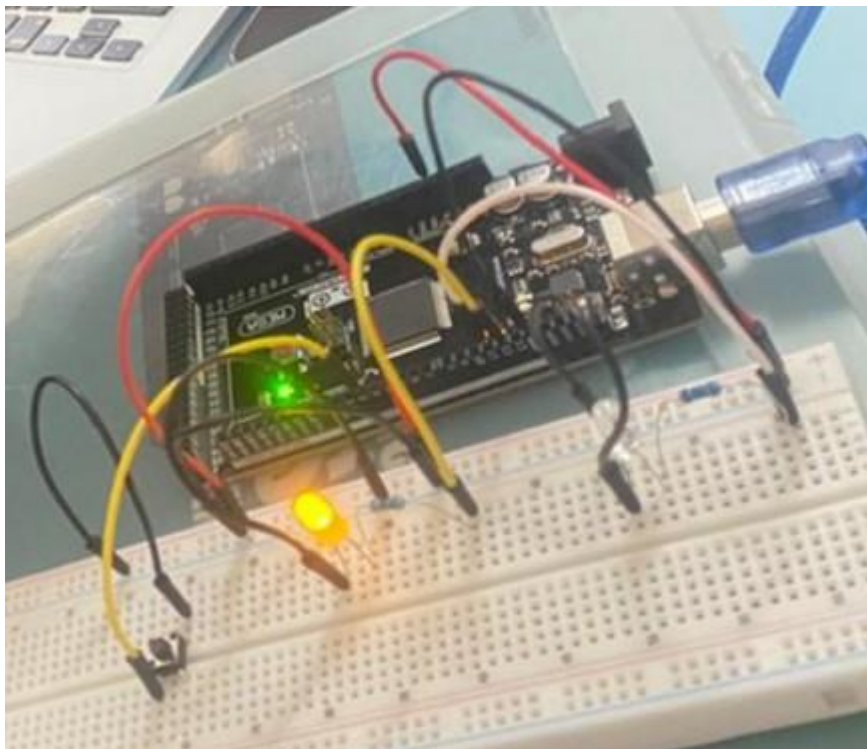
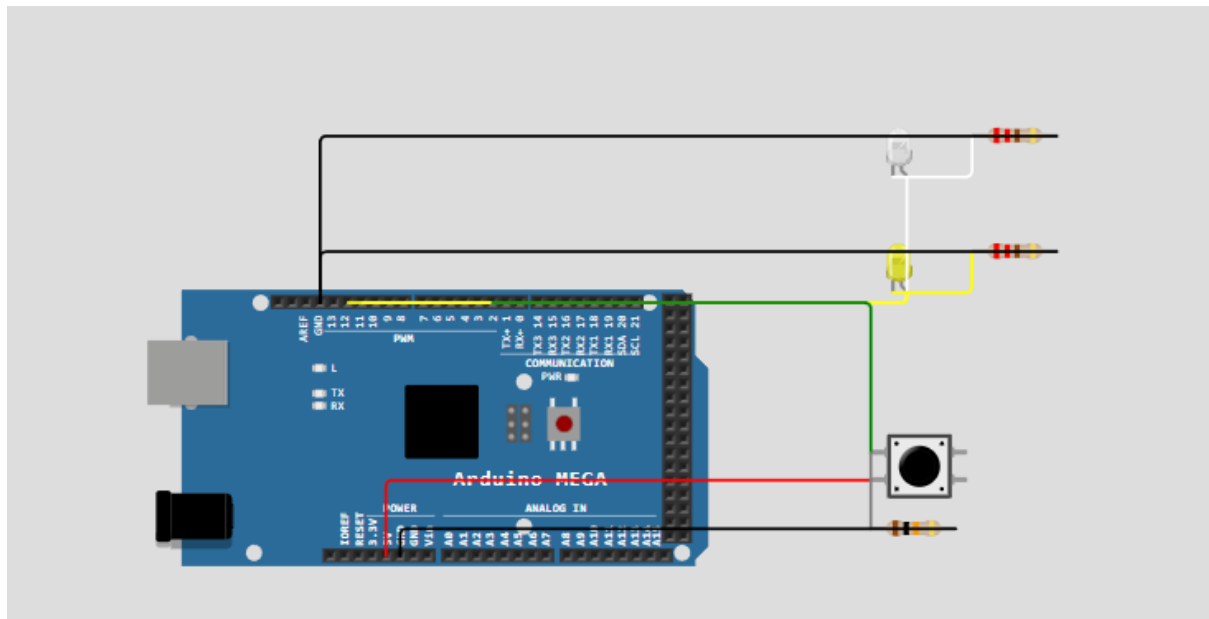
2.2. Affectation des ports

Port	Broche(s)	Fonction
PORT B	PB5 (OC1A)	Sortie PWM — LED blanche (Timer 1, canal A)
PORT B	PB6 (OC1B)	Sortie PWM — LED jaune (Timer 1, canal B)
PORT E	PE4	Entrée bouton poussoir (pull-up interne activé)
VCC / GND	Broches alimentation	Alimentation 5 V pour tous les périphériques

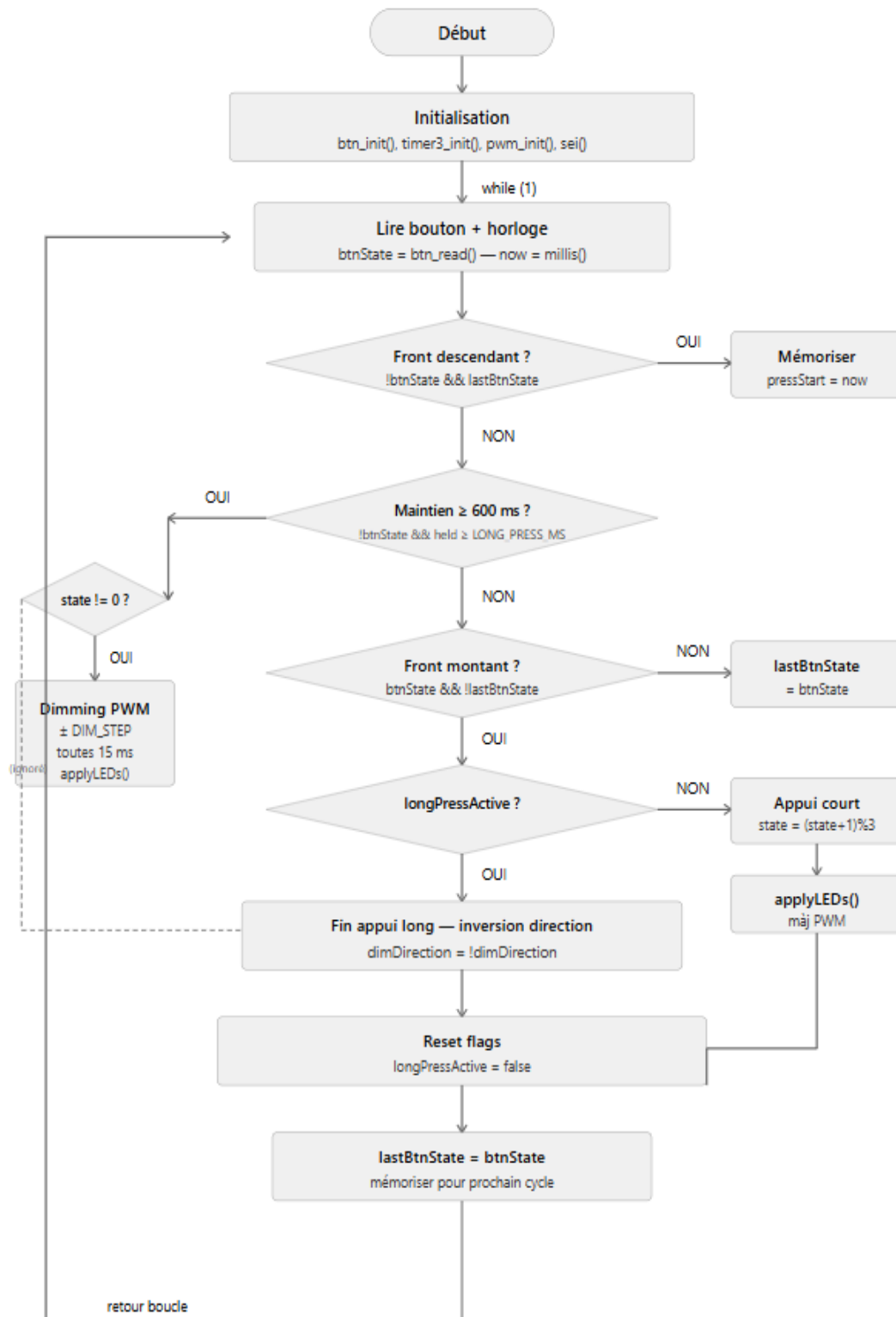
2.3. Environnement logiciel

- **Langage** : C standard (C99), compilé avec `avr-gcc`
- **IDE** : Microchip Studio (AVR Studio)
- **Bibliothèques** : `<avr/io.h>`, `<avr/interrupt.h>`, `<stdbool.h>`, `<stdint.h>`
- **Fréquence CPU** : `F_CPU = 16 MHz`

3. Montage



4. Logigramme



5. Déclarations des variables et constantes

Avant les fonctions, on déclare les constantes et variables globales qui pilotent le comportement du programme :

Variable / Constante	Type	Rôle
LONG_PRESS_MS	#define	Durée minimale (600 ms) pour déclencher un appui long
DIM_INTERVAL_MS	#define	Intervalle de temps (15 ms) entre deux pas de variation de luminosité
DIM_STEP	#define	Incrément/décroément de luminosité par pas (valeur 2)
BRIGHTNESS_MIN / MAX	#define	Limites de luminosité : 10 (min) et 255 (max) pour éviter l'extinction totale
ms_count	volatile uint32_t	Compteur de millisecondes incrémenté par l'ISR du Timer 3
state	int	État courant : 0 = éteint, 1 = LED blanche, 2 = LED jaune
brightness	int	Valeur PWM courante (10–255), conservée entre les changements d'état
dimDirection	bool	Direction de variation : true = augmente, false = diminue
lastBtnState / btnPressed	bool	Mémorisation de l'état précédent et en cours du bouton pour détecter les fronts
longPressActive	bool	Indique qu'un appui long est en cours (inhibe l'action courte au relâchement)
pressStart / lastDimTime	uint32_t	Horodatages pour mesurer la durée d'appui et cadencer le dimming

6. Gestion du PWM (Timer 1)

6.1. Initialisation pwm_init()

Le Timer 1 de l'ATmega2560 est configuré en mode Fast-PWM 8 bits avec un prescaler de 64. Les broches PB5 (OC1A, LED blanche) et PB6 (OC1B, LED jaune) sont placées en sortie :

```
DDRB |= (1<<PB5) | (1<<PB6); // PB5 et PB6 en sortie
TCCR1A = (1<<WGM10); // Fast-PWM 8-bit (WGM10 + WGM12)
TCCR1B = (1<<WGM12) | (1<<CS11) | (1<<CS10); // prescaler /64
OCR1A = 0; OCR1B = 0; // registres de comparaison à 0
```

Les bits COM1A1 et COM1B1 sont laissés à 0 au démarrage. Les sorties OC1A/OC1B sont ainsi déconnectées du timer et forcées à LOW, ce qui évite l'allumage résiduel des LEDs lorsqu'elles devraient être éteintes.

6.2. Fonctions `set_white()`, `set_yellow()`

Ces deux fonctions permettent d'activer ou de désactiver le signal PWM sur chaque LED de façon propre. Elles suivent le même principe :

```
void set_white(uint8_t val)
{
    if (val == 0)
    {
        TCCR1A &= ~(1 << COM1A1); // déconnecter OC1A
        PORTB &= ~(1 << PB5);     // forcer la broche LOW
    } else
    {
        OCR1A = val; // fixer la valeur PWM
        TCCR1A |= (1 << COM1A1); // connecter OC1A
        au timer
    }
}
```

Lorsque val vaut 0, on déconnecte la sortie OC1A du timer (bit COM1A1 remis à 0) et on force manuellement PB5 à 0 via PORTB. Cette approche garantit qu'aucun signal résiduel ne traverse la broche. La fonction `set_yellow()` fonctionne de manière identique pour OC1B (PB6).

7. Compteur millis() (Timer 3)

Le Timer 3 est configuré en mode CTC (Clear Timer on Compare Match) avec un prescaler de 64 et une valeur de comparaison OCR3A = 249. À une fréquence d'horloge de 16 MHz, cela génère une interruption exactement toutes les 1 ms :

```
// Fréquence interruption = F_CPU / (prescaler × (OCR3A + 1))
// = 16 000 000 / (64 × 250) = 1 000 Hz → 1 ms
```

À chaque déclenchement, l'ISR incrémente le compteur global `ms_count`. La fonction `millis()` retourne une copie atomique de ce compteur (interruptions désactivées pendant la lecture pour éviter une lecture incohérente d'un `uint32_t` sur 8 bits).

8. Lecture du bouton

Le bouton est connecté à la broche PE4 (pin 2 de la carte Mega). La fonction `btn_init()` configure PE4 en entrée et active la résistance de tirage interne (pull-up) :

```
DDRE &= ~(1 << PE4); // PE4 en entrée
PORTE |= (1 << PE4);  // pull-up interne activé
```

Avec un pull-up, la logique est inversée : la broche lit 1 lorsque le bouton est relâché et 0 lorsqu'il est enfoncé. La fonction `btn_read()` retourne `true` si le bouton est relâché, `false` s'il est appuyé. La détection des fronts (montant/descendant) est réalisée dans la boucle principale par comparaison avec `lastBtnState`.

9. Logique applicative

9.1. Appui court : changement d'état

Un appui court est détecté au moment du relâchement du bouton, à condition qu'aucun appui long n'ait été déclenché entre-temps (`longPressActive == false`). L'action effectuée est un simple passage à l'état suivant par modulo 3 :

```
state = (state + 1) % 3;
applyLEDs();
```

Les trois états possibles sont :

- État 0 : toutes les LEDs éteintes.
- État 1 : LED blanche allumée à la luminosité mémorisée, LED jaune éteinte.
- État 2 : LED jaune allumée à la luminosité mémorisée, LED blanche éteinte.

La luminosité est conservée d'un état à l'autre : si l'on réduit la LED blanche à 50 %, puis que l'on passe à la LED jaune, celle-ci s'allumera également à 50 %.

9.2. Appui long : variation de luminosité

Un appui long est détenu lorsque la durée d'appui dépasse LONG_PRESS_MS (600 ms) et que l'état n'est pas 0 (il serait inutile de varier la luminosité des LEDs éteintes). Le dimming est cadencé toutes les DIM_INTERVAL_MS (15 ms) :

```
if (!dimDirection)
{
    //descend
    brightness -= DIM_STEP;
    if (brightness <= BRIGHTNESS_MIN)
        brightness = BRIGHTNESS_MIN;
}
else
{
    //monte
    brightness += DIM_STEP;
    if (brightness >= BRIGHTNESS_MAX)
        brightness = BRIGHTNESS_MAX;
}
```

La direction de dimming (dimDirection) est réinitialisée à true (montée) à chaque nouveau front descendant du bouton. Ainsi, chaque appui long commence toujours par augmenter la luminosité, quelle que soit la direction du dimming précédent. La luminosité reste bornée entre BRIGHTNESS_MIN (10) et BRIGHTNESS_MAX (255) : la valeur minimale est volontairement supérieure à 0 pour garantir que la LED reste perceptiblement allumée.

10. Programme principal et boucle de mesure

La boucle principale détecte en permanence les fronts du bouton et applique la logique d'appui court / long sans utiliser d'interruptions externes ni de délai bloquant :

```
while (1)
{
    bool      btnState = btn_read();
    uint32_t  now      = millis();

    //Front descendant : début d'appui
    if (!btnState && lastBtnState)
    {
        pressStart      = now;
        btnPressed      = true;
        longPressActive = false;
    }
```



```

        lastDimTime      = now;
        //dimDirection est déjà fixée depuis le dernier relâchement
    }
    //Maintien : gestion appui long
    if (!btnState && btnPressed)
    {
        uint32_t held = now - pressStart;

        if (held >= LONG_PRESS_MS && state != 0)
        {
            longPressActive = true;
            if ((now - lastDimTime) >= DIM_INTERVAL_MS)
            {
                if (!dimDirection)
                {
                    //descend
                    brightness -= DIM_STEP;
                    if (brightness <= BRIGHTNESS_MIN)
                        brightness = BRIGHTNESS_MIN;
                }
                else
                {
                    //monte
                    brightness += DIM_STEP;
                    if (brightness >= BRIGHTNESS_MAX)
                        brightness = BRIGHTNESS_MAX;
                }
                applyLEDs();
                lastDimTime = now;
            }
        }
    }
    //Front montant : relâchement
    if (btnState && !lastBtnState && btnPressed)
    {
        btnPressed = false;
        if (!longPressActive)
        {
            //Appui court ? état suivant, luminosité conservée
            state = (state + 1) % 3;
            applyLEDs();
        }
        else
        {
            //Appui long terminé ? inverser pour le prochain appui long
            dimDirection = !dimDirection;
        }
        longPressActive = false;
    }
}

```

```
    lastBtnState = btnState;  
}
```

Cette architecture sans délai bloquant (`_delay_ms` dans la boucle principale) est essentielle : elle permet au programme de réagir immédiatement à tout changement d'état du bouton, y compris en cours de dimming.

11. Conclusion

Ce programme illustre plusieurs techniques fondamentales de la programmation embarquée sur microcontrôleur ATmega2560 :

- **Contrôle PWM matériel** : l'utilisation des registres COM1Ax permet de connecter ou déconnecter dynamiquement les sorties OC1A/OC1B du timer, évitant ainsi tout signal résiduel sur les LEDs.
- **Mesure de durée non bloquante** : le Timer 3 fournit une base de temps en millisecondes (`millis()`) utilisée pour mesurer la durée d'appui et cadencer le dimming sans figer la boucle principale.
- **Détection de fronts** : la comparaison entre l'état courant et l'état précédent du bouton (`lastBtnState`) permet de détecter fidèlement les événements de pression et de relâchement.
- **Machine à états** : le programme gère trois états (OFF, LED blanche, LED jaune) de façon propre grâce à une variable d'état et une fonction `applyLEDs()` centralisée.
- **Correction de comportement** : la réinitialisation de `dimDirection` à chaque nouvel appui long garantit une expérience utilisateur cohérente et prédictible.